

# 可持久化数据结构

可持久化数据结构就是用来查询某个历史版本中的信息，而不是查询第 $i$ 次修改之后的信息，所以这就需要一个数据结构不仅仅只记录当前信息，而应该记录所有的历史版本。

但是如果每个版本都重新开一个数据结构，那么消耗的空间为以前的 $m$ (修改次数)倍，很明显，这在时间空间上都是没有办法接受的，但是我们仔细研究就会发现，每次修改，只是改变了一部分信息，对于未修改的部分仍然和上个版本一样，那么我们就没有必要再重新新开一个数据结构，而是根据上一个版本的数据结构，未修改的地方保持原样，修改的地方重新开点，这样的操作方式每次最多新开 $\log n$ 个点，时间复杂度和空间复杂度都过得去。

所有的可持久化数据结构都差不多是这个原理，比如可持久化线段树，可持久化平衡树，01trie...

# 可持久化线段树

可持久化线段树又名主席树，在以前就接触过，当时用来求解区间第 $k$ 大，但是对可持久化并没有太大的感受，下面我们重新来看一题。

如题，你需要维护这样的一个长度为 $N$ 的数组，支持如下几种操作：

- 1：在某个历史版本上修改某一个位置上的值
- 2：访问某个历史版本上的某一位置的值

此外，每进行一次操作（对于操作2，即为生成一个完全一样的版本，不作任何改动），就会生成一个新的版本。版本编号即为当前操作的编号（从1开始编号，版本0表示初始状态数组）

# 可持久化线段树

这是一道主席的裸题，做法和以前写过的主席树一样。

```
int build(int l,int r)
{
    int now=++k;
    if(l==r)
    {
        tree[now].val=a[l];
        return now;
    }
    int mid=(l+r)/2;
    tree[now].l=build(l,mid);
    tree[now].r=build(mid+1,r);
    return now;
}
```

# 可持久化线段树

```
int query(int pre,int l,int r,int x)
{
    if(l==r)
        return tree[pre].val;
    int mid=(l+r)/2;
    if(x<=mid)
        return query(tree[pre].l,l,mid,x);
    else
        return query(tree[pre].r,mid+1,r,x);
}
```

同样，区间修改，区间查询也是相同的做法。

# 可持久化并查集

学习了可持久化线段树之和，我们可以引申出可持久化并查集，也就是合并第 $k$ 操作之后的两个集合。实际上也就是用可持久化线段树+并查集处理。

$n$ 个集合  $m$ 个操作

操作：

- 1  $a$   $b$  合并 $a, b$ 所在集合
- 2  $k$  回到第 $k$ 次操作之后的状态(查询算作操作)
- 3  $a$   $b$  询问 $a, b$ 是否属于同一集合，是则输出1否则输出0

# 可持久化线并查集

其实可持久化并查集可以用可持久化线段树+并查集来维护，对于线段树的每一个根结点，记录该点的父亲结点的位置，初始时都为他本身，如果有两个集合合并，就把其中一个的祖先更新为另一个。

我们平时使用的并查集有路径压缩，但是如果涉及可持久化，那么路径压缩之后就不能还原了，所以这里的并查集不能路径压缩，但是不压缩的并查集很有可能会特别慢，所以我们要使用并查集的启发式合并。

其实原理也很简单，深度浅的集合向深度深的集合合并，保证每次合并之后集合深度没太大变化。

比如A集合深度为3，B集合深度为5，那么把A集合的祖先合并到B集合的祖先上去，那么合并之后的B集合的深度仍然为5，但是反过来，合并之后的集合深度为6。

# 可持久化并查集

## 第一步：建树

```
int build(int l,int r)
{
    int now=++k;
    if(l==r)
    {
        tree[now].fa=l; //now这个点的父亲结点的编号,
        return now;
    }
    int mid=(l+r)/2;
    tree[now].l=build(l,mid);
    tree[now].r=build(mid+1,r);
    return now;
}
```

# 可持久化并查集

## 第二步：找祖先

```
int query(int now,int l,int r,int x)
{
    if(l==r)
        return now;//返回结点信息，不要直接返回父亲结点
    int mid=(l+r)/2;
    if(x<=mid)
        return query(tree[now].l,l,mid,x);
    else
        return query(tree[now].r,mid+1,r,x);
}
int getf(int now,int x)
{
    int fx=query(now,1,n,x);
    if(x==tree[fx].fa)//如果该点就是祖先
        return fx;
    else
        return getf(now,tree[fx].fa);//可持久化并查集不可以路径压缩
}
```

# 可持久化并查集

## 第三步：更新

```
int update(int pre,int l,int r,int x,int y)
{
    int now=++k;
    tree[now]=tree[pre];
    if(l==r)
    {
        tree[now].fa=y;
        deep[now]=deep[pre]; // 继承深度
        return now;
    }
    int mid=(l+r)/2;
    if(x<=mid)
        tree[now].l=update(tree[pre].l,l,mid,x,y);
    else
        tree[now].r=update(tree[pre].r,mid+1,r,x,y);
    return now;
}
```

# 可持久化并查集

## 第四步：合并操作

```
if(opt==1)//合并
{
    scanf("%d%d",&x,&y);
    int f1=getf(root[i-1],x);//找到父亲节点在线段树中结点编号
    int f2=getf(root[i-1],y);
    if(tree[f1].fa==tree[f2].fa)//不需要合并
    {
        root[i]=root[i-1];
        continue;
    }
    if(deep[f1]>deep[f2])
    {
        int tmp=f1;f1=f2;f2=tmp;
    }
    root[i]=update(root[i-1],1,n,tree[f1].fa,tree[f2].fa);
    if(deep[f1]==deep[f2])
        add(root[i],1,n,tree[f2].fa);
    //如果两个并查集深度一样，更新合并之后并查集深度+1
}
```

# 可持久化并查集

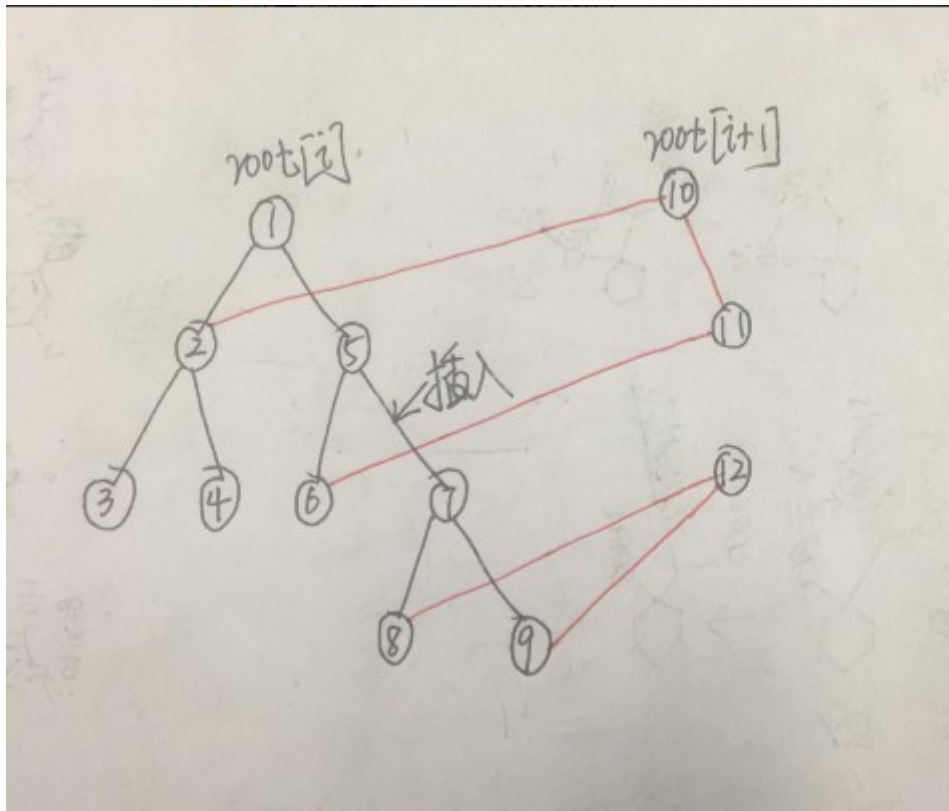
第五步：回溯，询问

```
else if(opt==2)//返回
{
    scanf("%d",&x);
    root[i]=root[x];
}
else if(opt==3)
{
    scanf("%d%d",&x,&y);
    root[i]=root[i-1];
    int f1=getf(root[i-1],x);
    int f2=getf(root[i-1],y);
    if(tree[f1].fa==tree[f2].fa)
        printf("1\n");
    else
        printf("0\n");
}
```

# 可持久化平衡树

既然学会了可持久化线段树，那么可持久化平衡树其实就是一样的。先建一个主树，然后复制一棵树(只复制根结点)，再在主树的基础上进行修改，都是一个套路。但是要注意通过旋转维持平衡的平衡树是很难可持久化的，因为树的形态完全改变了，所以支持可持久化的平衡树是非旋平衡树——FHQ Treap。对于FHQ中的两个核心操作split和merge，他并不是把上一个版本的树给split和merge了，而是每次都是新建结点来实现的。比如分裂，先复制断开的两个结点，然后再把复制的两个结点分裂。

# 可持久化平衡树



# 可持久化平衡树

## 一：分裂

```
void split(int u,int x,int &l,int &r)
{
    if(!u){
        l=r=0;return ;
    }
    if(tree[u].val<=x){
        l=++k;//左子树为根,新树的根结点
        tree[l]=tree[u];//先继承里面的所有信息
        split(tree[l].r,x,tree[l].r,r);
        update(l);
    }
    else{
        r=++k;
        tree[r]=tree[u];
        split(tree[r].l,x,l,tree[r].l);
        update(r);
    }//不更新u, u这棵树是不变的
}
```

# 可持久化平衡树

## 二：合并

```
int merge(int x,int y)
{
    if(!x||!y)
        return x+y;
    if(tree[x].key<tree[y].key)//左子树为根
    {
        int now=++k;
        tree[now]=tree[x];
        tree[now].r=merge(tree[now].r,y);
        update(now);
        return now;
    }
    else{
        int now=++k;
        tree[now]=tree[y];
        tree[now].l=merge(x,tree[now].l);
        update(now);
        return now;
    }
}
```

# 可持久化平衡树

## 三：插入，删除

```
scanf("%d%d%d",&pre,&opt,&x);
root[i]=root[pre];//先继承上一个版本
if(opt==1)//插入数x
{
    split(root[i],x,l,r);
    p=add(x);
    root[i]=merge(merge(l,p),r);//一定要按顺序合并
    //不能merge(merge(l,r),p)
}
else if(opt==2)//删除函数
{
    split(root[i],x,l,r);
    split(l,x-1,l,p);
    p=merge(tree[p].l,tree[p].r);
    root[i]=merge(merge(l,p),r);
}
```



# 可持久化文艺平衡树

带区间操作的可持久化平衡树，写法和平衡树的可持久化一样。对于懒标记，如果要下传，新开两个点传下去就可以了。



# 可持久化文艺平衡树

```
void pushdown(int u)
{
    if(!tree[u].flag) return ;
    if(tree[u].l)//如果左儿子存在
    {
        int now=++k;
        tree[now]=tree[tree[u].l];//复制, 不能影响以前的
        tree[u].l=now;
    }
    if(tree[u].r)
    {
        int now=++k;
        tree[now]=tree[tree[u].r];
        tree[u].r=now;
    }
    int tmp=tree[u].l;tree[u].l=tree[u].r;tree[u].r=tmp;
    if(tree[u].l) tree[tree[u].l].flag^=1;
    if(tree[u].r) tree[tree[u].r].flag^=1;
    tree[u].flag=0;
}
```

# 可持久化文艺平衡树

```
void split(int u,int x,int &l,int &r)
{
    if(!u) {
        l=r=0;return ;
    }
    pushdown(u);
    if(tree[tree[u].l].size<x)//在右子树
    {
        l=++k;
        tree[l]=tree[u];
        split(tree[l].r,x-tree[tree[u].l].size-1,tree[l].r,r);
        pushup(l);
    }
    else//在左子树
    {
        r=++k;
        tree[r]=tree[u];
        split(tree[r].l,x,l,tree[r].l);
        pushup(r);
    }
}
```

# 可持久化文艺平衡树

```
int merge(int x,int y)
{
    if(!x||!y)
        return x+y;
    pushdown(x);
    pushdown(y);
    if(tree[x].key<tree[y].key)//x为根
    {
        tree[x].r=merge(tree[x].r,y);
        pushup(x);
        return x;
    }
    else
    {
        tree[y].l=merge(x,tree[y].l);
        pushup(y);
        return y;
    }
}
```



# 练习题

洛谷 P3834: 可持久化线段树

洛谷 P3919: 可持久化数组

洛谷 P3835: 可持久化平衡树

洛谷 P3169: 任务查询系统

洛谷 P3293: 美味

洛谷 P3567: KUR

洛谷 P5055: 可持久化文艺平衡树

洛谷 P3402: 可持久化并查集

